



PROJECT-BASED LEARNING

PHASE-I EVALUATION

Adaptive Memory Allocation Fragmentation Analyzer

Team ID: DSCPP-III-2026-T012

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026-27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T012

Semester: 3rd

Section: A,E,F,H

Domain: C++,DSA

Team Members

1. *Vamsi -2510011373*
2. *Shweta Singhal -2510011209*
3. *Akshat Kumar-2510011779*
4. *Parik Maan -2510010904*

Mentor

Mr. Himanshu Mainwal

Interactions completed: _____

Problem Statement & Motivation

Problem Statement

Operating systems frequently face memory fragmentation and inefficient allocation due to repeated process allocation and deallocation. This project simulates memory management and compares First Fit, Best Fit, and Worst Fit to improve memory utilization and reduce allocation failures.

Motivation

- ▶ *Current Gap: Existing simulations analyze allocation strategies separately.*
- ▶ *Memory Inefficiency: Fragmentation reduces utilization and may cause allocation failures.*
- ▶ *Adaptive Need: Compare strategies, select the best for each workload.*

Target Users / Stakeholders

- ▶ *Primary Users: Students and educators studying memory management.*
- ▶ *Secondary Stakeholders: Developers and researchers working on memory allocation.*
- ▶ *Expected Benefit: Easy comparison and analysis of allocation strategies.*

Objectives

Project Objectives

1. *Develop a configurable virtual memory simulation environment.*
2. *Implement First Fit, Best Fit, and Worst Fit strategies.*
3. *Manage processes using memory, arrival, execution time, and priority.*
4. *Implement memory allocation, deallocation, splitting, and merging.*
5. *Detect and measure internal and external fragmentation.*
6. *Measure memory utilization, free memory, and allocation failures..*
7. *Compare allocation strategies using common workloads.*
8. *Develop an Adaptive Engine for workload-based strategy selection.*
9. *Generate memory statistics and performance reports..*

Key Objective: *Develop an adaptive memory allocation system that improves memory utilization and reduces fragmentation.*

Proposed Solution

Proposed Approach

1. *Workload Input – Enter or generate process and memory data.*
2. *Memory Simulation – Simulate allocation and deallocation.*
3. *Allocation Algorithms – Apply First Fit, Best Fit, and Worst Fit.*
4. *Fragmentation Analysis – Measure utilization, free memory, and fragmentation.*
5. *Adaptive Selection – Compare results and select a suitable strategy.*

Key Features

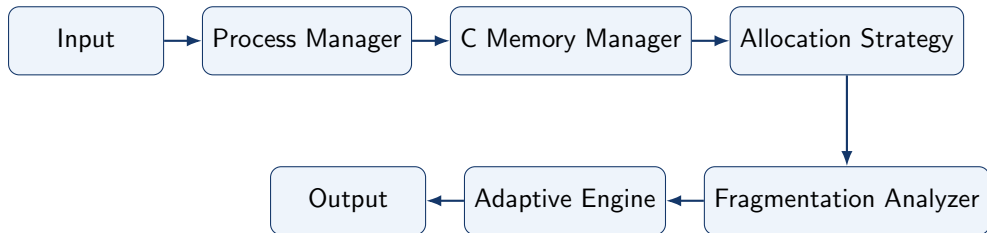
- ▶ *Virtual Memory Simulation*
- ▶ *Multiple Allocation Strategies*
- ▶ *Process and Memory Management*
- ▶ *Fragmentation Analysis*
- ▶ *Adaptive Strategy Selection*

Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C	Arrays / Vectors, Linked Lists, Queues, Priority Queue / Heap	Process & Memory Management
Algorithms	Hashing, Searching, Sorting, Greedy Algorithms	Allocation Strategy Analysis
OOPs with C++	Classes, Objects, Encapsulation, Inheritance, Polymorphism	Allocation & Adaptive Engine
C++ Features	STL, File Handling, Exception Handling, Modular Design	Simulation & Reporting
Operating Systems	Processes, Memory Management, Allocation, Deallocation, Fragmentation	Memory Simulation & Analyzer
DBMS / Data Management	File Handling, Data Storage, Basic Data Management	Results & Reports

Key Integration: DSA + C++ + Operating Systems concepts are combined to simulate, analyze, and adapt memory allocation.

System Architecture / Workflow



Major Components

- ▶ *Process Manager*
- ▶ *Memory Allocation Module*
- ▶ *Fragmentation Analyzer*

Data / Control Flow

- ▶ *Input process and memory request*
- ▶ *Allocation and fragmentation analysis*
- ▶ *Adaptive allocation result*

Technology Stack & Methodology

Technology Stack

- ▶ Programming: *C / C++*
- ▶ Database: *File Handling (.txt files)*
- ▶ Tools / Framework: *Standard C/C++ Libraries, STL*
- ▶ IDE: *Code::Blocks / Dev-C++/VS Code*
- ▶ Version Control: *Git / GitHub*

Development Methodology

1. Requirement Analysis – Identify system requirements
2. System Design– Design modules and workflow
3. Implementation– Develop allocation algorithms
4. Testing– Test allocation and fragmentation cases
5. Evaluation– Compare performance and memory utilization

Justify the selection of technologies and methodology.

Initial Progress & Team Contribution

Progress Achieved

- ▶ Background and literature study
- ▶ Basic requirements identified
- ▶ Initial system architecture prepared
- ▶ Project structure and modules planned

Individual Contribution

Member	Role	Contribution
M1 - Vamsi	Lead	Project planning, architecture, core logic
M2 - Shweta	Developer	Memory allocation algorithms and module development
M3 - Akshat	Testing	Test cases, data preparation, result validation
M4 - Parik	Developer/Docs	Fragmentation analysis, documentation and presentation

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: Proposal & Design— Requirements, architecture, and algorithm selection
2. Phase-II: Development— Memory allocation and fragmentation modules
3. Phase-III: Final Implementation— Integration, testing, optimization, and final results

Expected Outcomes

- ▶ *Working memory allocation simulator*
- ▶ *Fragmentation analysis with measurable results*
- ▶ *Comparison of allocation strategies*
- ▶ *Practical understanding of memory management*
- ▶ *Scope for future optimization and scalability*

References

1. *GeeksforGeeks – Operating System Memory Management concepts*
2. *W3Schools – C Programming Memory Management tutorials*
3. *Programming in ANSI C – E. Balagurusamy, McGraw Hill*
4. *Data Structures Through C – G.S. Baluja*

Thank You

Questions & Suggestions